0xCommit

Security Audit Report

DZap

Version: Final

Date: 3rd Sept 2025

Table of Contents

License

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

Introduction

Purpose of this report

0xCommit has been engaged by **DZap -Contracts** to perform a security audit of several Solana Programs components.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine solana program bugs, which might lead to unexpected behaviour.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebases Submitted for the Audit

The audit has been performed on the following GitHub repositories:

Version	Commit ID
Final	**bd0bf62f97faf4be7c6cf0434d1f3211b0a84e6d**

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Overview

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line by line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Summary of Findings

Sr. No.	Description	Severity	Status
1	Insufficient Validation in Token Output Processing	Low	Acknowledged
2	Gas Optimization and DoS Vectors	Low	Acknowledged
3	Inconsistent Error Handling	Low	Acknowledged
5	Missing Natspec Documentation	Low	Acknowledged

Detailed Findings

1. Insufficient Validation in Token Output Processing

Location: `DZapTokenHandler.sol` - `_handleERC20Output`, `_handleNativeOutput`

Description:
The output token handling functions calculate return amounts based on balance differences without considering external factors that could manipulate these balances. Fee calculations can underflow if not properly validated.

Impact:

- Integer underflow in fee calculations
- Incorrect token distribution
- Loss of user funds through manipulated balances

```
function _handleERC20Output(OutputToken calldata _outputToken, uint256 _initialBalance) pr
    address recipient = _getRecipient(_outputToken);
    uint256 currentBalance = LibAsset.getBalance(_outputToken.tokenAddress, recipient);

    require(currentBalance >= _initialBalance, "Balance decreased");
    uint256 returnAmount = currentBalance - _initialBalance;

    if (returnAmount < _outputToken.minReturn) {
        revert InvalidReturnAmount(returnAmount, _outputToken.minReturn);
    }

    if (_outputToken.transferType == OutputTransferType.ReceiveAndTransfer) {
        require(returnAmount >= _outputToken.feeAmount, "Insufficient return for fees");
        uint256 transferAmount = returnAmount - _outputToken.feeAmount;
        require(transferAmount > 0, "Zero transfer amount");
        LibAsset.transferERC20(_outputToken.tokenAddress, _outputToken.recipient, transfer
    }
}
```

Acknowledged

2. Gas Optimization and DoS Vectors

Location: Multiple locations with unbounded loops

Description:
Several functions contain unbounded loops that could lead to out-of-gas errors:

- _processInputTokens loops through all input tokens
- _handleZap loops through all zap data
- whitelistAdapters loops through adapter array

Impact:

- Transaction failures due to gas limits
- Potential DoS by providing large arrays
- Poor user experience with failed transactions

Remediation:

Add array length limits:

```
uint256 public constant MAX_ARRAY_LENGTH = 50;

function _handleZap(ZapData[] calldata _zapData, address _user) internal {
    uint256 length = _zapData.length;
    require(length <= MAX_ARRAY_LENGTH, "Array too long");

    for (uint256 i; i < length; ++i) {
        // Process with gas checks
        require(gasleft() > 50000, "Insufficient gas");
        // ... rest of logic
    }
}
```

3. Inconsistent Error Handling

Location: Various locations

Description:
Mix of require statements and custom errors, inconsistent error messages.

Impact:

- Higher gas costs with require strings
- Inconsistent developer experience
- Difficult debugging

Remediation:
Convert all requires to custom errors:

```
error ArrayLengthMismatch(uint256 provided, uint256 expected);
error InsufficientGas(uint256 available, uint256 required);
error InvalidTokenContract(address token);
```

Acknowledged

4. Missing Natspec Documentation

Location: Several internal functions

Description:
Incomplete documentation for internal functions and state variables.

Impact:

- Reduced code maintainability
- Difficult onboarding for new developers
- Potential for misuse of functions

Remediation:
Add complete Natspec:

```
/// @notice Processes input tokens for a zap operation
/// @dev Handles multiple token types with different transfer mechanisms
/// @param _inputTokens Array of input token specifications
/// @param _user Address of the user providing tokens
/// @param _spender Address that will receive approvals
/// @return needsErc1155Revoke Whether ERC1155 approvals need revocation
```

Acknowledged